



Document: *EasyLib_Sviluppo*

Revision: *x.x*



Dipartimento di Ingegneria e Scienza
dell'Informazione

Progetto:

EasyLib

Titolo del documento:

Sviluppo

Document Info

Doc. Name	D2-EasyLib_Sviluppo	Doc. Number	D3 V1.x
Description	Documento di sviluppo dell'applicazione		

INDICE

Sommario

Scopo del documento	3
1. Web APIs	4
2. Implementation	7
2.1. Repository Organization	7
2.2. Branching strategy e organizzazione del lavoro	7
2.3. Dependencies	8
2.4. Database	8
2.5. Testing	8
3. FrontEnd	10
4. Deployment	11

Scopo del documento

Il presente documento riporta tutte le informazioni necessarie per lo sviluppo di una parte dell'applicazione EasyLib. In particolare, presenta tutti gli artefatti necessari per realizzare i servizi di gestione dei libri e dei prestiti con l'applicazione EasyLib.

Partendo dalla presentazione delle web APIs del servizio web, proseguendo con l'organizzazione del codice, il modello dati, e ulteriori informazioni su testing. Infine, una sezione dedicata al front-end e una agli aspetti di deployment.

1. Web APIs

Le api sono state documentate secondo le specifiche openapi3 e la documentazione è consultabile pubblicamente su [swagger/apiary/altro](https://easylib.docs.apiary.io/#) al seguente indirizzo <https://easylib.docs.apiary.io/#>.

<<<Ulteriori note su scelte di design delle api>>>

La specifica delle API è disponibile nel repository al link <https://github.com/unitn-software-engineering/EasyLib/blob/master/oas3.yaml>. Il contenuto del file .yaml è qui riportato per esteso:

```
openapi: 3.0.0
info:
  version: '1.0'
  title: "EasyLib OpenAPI 3.0"
  description: API for managing book lendings.
  license:
    name: MIT
servers:
  - url: http://localhost:8000/api/v1
    description: Localhost
paths:
  /students:
    post:
      description: >-
        Creates a new student in the system.
      summary: Register a new student
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required:
                - email
              properties:
                email:
                  type: string
                  description: 'Email address of the student'
      responses:
        '201':
          description: 'User created. Link in the Location header'
          headers:
            'Location':
              schema:
                type: string
              description: Link to the newly created student.
  /books:
    get:
      description: >-
```

```
Gets the list of books.
It is possible to show users by their role /users?role={role}
summary: View all books
parameters:
- in: query
  name: role
  schema:
    type: string
    enum: [user, poweruser, admin]
responses:
'200':
  description: 'Collection of books'
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/Book'
/booklendings:
post:
  description: >-
    Creates a new booklending.
  summary: Borrow a book
  responses:
'201':
  description: 'Booklending created. Link in the Location header'
  headers:
    'Location':
      schema:
        type: string
        description: Link to the newly created booklending.
  requestBody:
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Booklending'

components:
  schemas:
    Student:
      type: object
      required:
        - id
        - email
      properties:
        id:
          type: integer
          description: 'ID of the user'
        email:
          type: string
          description: 'Email address of the user'
    Book:
      type: object
      required:
        - title
```

```
- author
- ISBN
- status
properties:
  title:
    type: string
    description: 'Title of the book'
  author:
    type: string
    description: 'Author of the book'
  ISBN:
    type: string
    description: 'ISBN of the book'
  status:
    type: string
    enum: [available, lendend]
    description: 'Tells whether the book is currently available or not'
Booklending:
  type: object
  required:
    - student
    - book
  properties:
    user:
      type: string
      description: 'Link to the user'
    book:
      type: integer
      description: 'Link to the book'
```

2. Implementation

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie: NodeJS e VueJS, per la gestione dei dati abbiamo utilizzato MongoDB. Lo stack tecnologico è stato motivato dal materiale fornito nel corso / conoscenze pregresse / altro

2.1. Repository Organization

Il codice del progetto (<https://github.com/unitn-software-engineering/EasyLib>) è organizzato secondo la seguente struttura:

- /src
 - /api endpoint per le risorse studenti e prestiti
 - /middleware middleware di autenticazione
 - /models modelli dati mongoose
 - app.js applicazione Express.js
- /frontend codice per la parte del front-end
- /doc
 - openapi3.yaml documentazione api
- package.json file di configurazione del progetto npm
- .gitignore configurazione repository git
- .env.example esempio di file di configurazione variabili d'ambiente

2.2. Branching strategy e organizzazione del lavoro

Lo sviluppo ha visto una collaborazione attiva tra i membri del gruppo. Ci siamo divisi il lavoro in questo modo: ... In totale abbiamo eseguito più di xx commit distribuiti tra i membri in questo modo Lo sbilanciamento tra il numero di commit è dovuto a errori/codice di librerie esterne erroneamente committato/alto numero di piccoli commit vs numero di commit contenuto ma commit molto consistenti...

A livello di repository abbiamo optato per una strategia di branching GitFlow Workflow. Durante lo sviluppo di nuove funzionalità abbiamo lavorato sui seguenti branch, sempre derivandoli dal branch develop:

- "books": il branch è stato usato per sviluppare il modello dati e le api relative alla risorsa libri;
- "home_page": il branch è stato dedicato allo sviluppo della prima pagina di benvenuto del sito;
- ...

2.3. Dependencies

Il progetto npm si basa sui seguenti moduli esterni:

- | | |
|-----------------------|---|
| - Cors | Per il supporto alle chiamate cors |
| - Express | Framework web per il backend |
| - Google-auth-library | Supporto al login google |
| - Jsonwebtoken | Autenticazione JWT |
| - Mongoose | Libreria per interfacciarsi con mongoDB |

E le seguenti dipendenze di sviluppo:

- | | |
|-------------|---|
| - Dotenv | Gestione variabili d'ambiente in ambiente dev da file.env |
| - Jest | Testing |
| - Supertest | Testing endpoint express |

2.4. Database

Per la gestione dei dati utili all'applicazione abbiamo definito due principali strutture dati tramite l'utilizzo di mongoose.

Students: ...

Booklendings: ...

2.5. Testing

Le api e il relativo codice presentano una test-suite che consente di verificarne il corretto funzionamento. I test sono utilizzata all'interno della configurazione di CI/CD.

L'implementazione dei test è organizzata in file .test.js che affiancano i vari file dell'implementazione / I test sono implementati tutti in un'unica cartella ...

Dei xxx casi di test che sono stati definiti, yyy sono stati implementati.

N. Test Case	Descrizione Test Case	Test Data	Precondizioni	Dipendenze	Risultato Atteso	Risultato riscontrato	Note
1	Creazione di un'account in modo corretto	<username> non vuota <password> rispettosa delle politiche di sicurezza	<username> mai inserita prima nel sistema	---	Viene creata l'account specificato. Il sistema risponde con <msg OK>		
1.1	Creazione di un'account specificando uno username già esistente	<username> già inserita nel sistema	<username> già inserita nel sistema	Questo caso di test deve essere fatto dopo il caso di testo numero 1 specificando la stessa <username>	Viene mostrato un messaggio di errore <error msg user exists>, l'account non viene creato ed il sistema mostra alcuni username disponibili da utilizzare		
2	Creazione di un'account non specificando lo username	<username> Vuota	---	---	Viene mostrato <errore msg empty user> e l'account non viene creato		
3	Creazione di account violando le politiche di sicurezza (password banale, conferma password errata, ...)	<password> non rispettosa delle politiche di sicurezza	---	---	Viene mostrato un messaggio di errore e l'account non viene creato		

3. FrontEnd

Il FrontEnd fornisce le funzionalità di visualizzazione, inserimento e cancellazione dei dati dell'applicazione EasyLib. In particolare, l'applicazione è composta da una Home Page, da una pagina per la gestione dei libri e una pagina per la gestione dei prestiti.

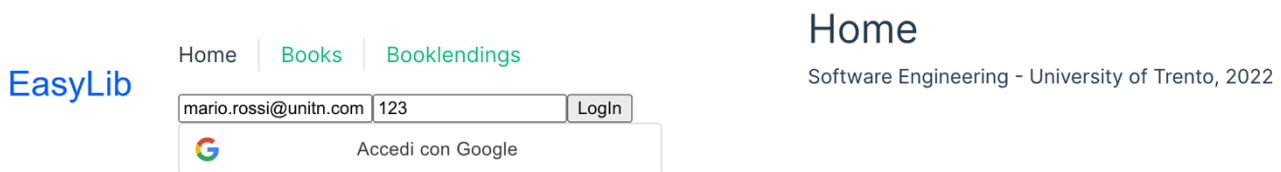


Figura 1 Home Page

In Figura 1 è mostrata l'home page dell'applicazione. Da qui l'utente può autenticarsi e navigare nelle altre pagine dell'app.

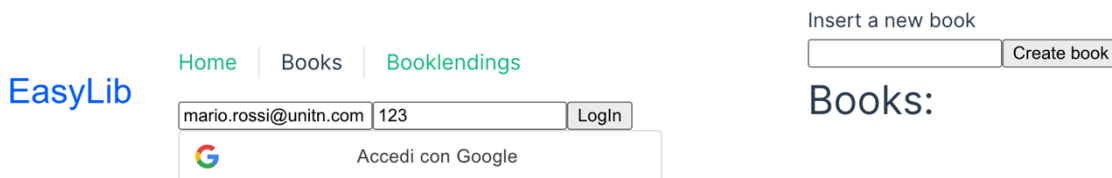


Figura 2 Schermata libreria

In Figura 2 è mostrata lista dei libri prenotabili dall'utente. Inoltre è possibile inserire nuovi libri specificandone il titolo.

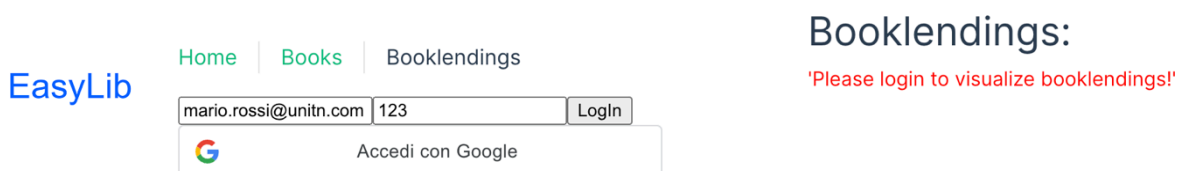


Figura 3 Schermata libri in prestito

In Figura 3 è visibile la lista dei libri attualmente in prestito dall'utente. Come utente non loggato ricevo un messaggio di errore.

4. Deployment

Un'istanza del backend è ospitata sulla piattaforma render.com ed è disponibile al link <https://easy-lib.onrender.com/>. <<<altri eventuali link di deploy>>> Il frontend è deployato separatamente al link <https://easylibvue.onrender.com/>.

La configurazione CI/CD basata sulle GitHub Actions esegue automaticamente il deploy del branch main quando vengono superati con successo tutti i test. <<<altri dettagli>>>

Per accedere al deploy dell'applicazione è possibile utilizzare i seguenti account:

- Utente studente
 - Username: mario.rossi@unitn.it Password: 123
- Altra tipologia di utente
 - ...

Per problemi con il deploy contattare <<< mail amministratore / mail di tutti i membri >>>